

Understanding mobile GUI: From pixel-words to screen-sentences

Jingwen Fu^{a,1}, Xiaoyi Zhang^{b,1}, Yuwang Wang^c, Wenjun Zeng^d, Nanning Zheng^{a,*}

^a National Key Laboratory of Human-Machine Hybrid Augmented Intelligence, National Engineering Research Center for Visual Information and Applications, and Institute of Artificial Intelligence and Robotics, Xi'an Jiaotong University, China

^b Microsoft Research Asia, No. 5 Dan Ling Street, Beijing, 100080, China

^c Tsinghua University, No 30, Shuangqing road, Beijing, 100084, China

^d EIT, No. 568, Tongxin Road, Zhuangshi Street, Zhenhai District, Ningbo, China

ARTICLE INFO

Communicated by X. Lu

Keywords:

GUI understanding
Transformer
Detection

ABSTRACT

The ubiquity of mobile phones makes mobile GUI understanding an important task. Most previous works in this domain require human-created metadata of screens (e.g. View Hierarchy) during inference, which unfortunately is often not available or reliable enough for GUI understanding. Inspired by the impressive success of Transformers in NLP tasks, targeting for purely vision-based GUI understanding, we extend the concepts of *Words/Sentence* to *Pixel-Words/Screen-Sentence*, and propose a mobile GUI understanding architecture: *Pixel-Words* to *Screen-Sentence* (PW2SS). In analogy to the individual *Words*, we define the *Pixel-Words* as **atomic** visual components (text and graphic components), which are visually consistent and semantically clear across screenshots of a large variety of design styles. The *Pixel-Words* extracted from a screenshot are aggregated into *Screen-Sentence* with a Screen Transformer proposed to model their relations. Since the *Pixel-Words* are defined as atomic visual components, the ambiguity between their visual appearance and semantics is dramatically reduced. We are able to make use of metadata available in training data to auto-generate high-quality annotations for *Pixel-Words*. A dataset, RICO-PW, of screenshots with *Pixel-Words* annotations is built based on the public RICO dataset, which will be released to help to address the lack of high-quality training data in this area. We train a detector to extract *Pixel-Words* from screenshots on this dataset and achieve metadata-free GUI understanding during inference. We conduct experiments and show that *Pixel-Words* can be well extracted on RICO-PW and well generalized to a new dataset, P2S-UI, collected by ourselves. The effectiveness of PW2SS is further verified in the GUI understanding tasks including relation prediction, clickability prediction, screen retrieval, and app type classification.

1. Introduction

As mobile phones have become indispensable for human daily life, understanding GUI becomes a very important capability for AI to accomplish such tasks as language navigation [1], task automation [2,3], reverse software engineering [4,5], screen reader [6], etc. The metadata of screenshot, e.g. View Hierarchy (VH), provides a tree structured description of the UI components forming the screenshot. Most previous works [1,2,7] rely on the metadata to understand GUI. Unfortunately, the metadata is often noisy [3,8,9] due to the large varieties of platforms, third-party UI toolkits, coding styles, etc. [3]. What is worse is that the metadata is often not accessible due to privacy or compliance issues. Understanding the GUI from the screenshot only is a challenging and less-studied area. The first challenge is the complexity of screenshots. Different from natural images, the screenshots

consist of UI components of a large number of categories. The UI components from different categories can be visually similar, and complex UI components may be composed of other simpler UI components, e.g., list views can be decomposed into icons and texts. The visual appearances of UI components vary due to the large variety of UI design styles. Chen et al. [10] try to detect the non-text UI components from screenshots with traditional low-level vision algorithms, e.g. boundary extraction, but they lack semantic understanding and are not robust on screenshots with complex layouts. Zhang et al. [3] manually annotate the UI components of a self-collected private dataset and train their detector to detect those UI components. Both works are limited to only detecting UI components from screenshots, without screen-level understanding. Another challenge is the lack of high-quality annotated large scale datasets, which limits deep learning based methods. Liu

* Corresponding author.

E-mail addresses: fu1371252069@stu.xjtu.edu.cn (J. Fu), xiaoyizhang@microsoft.com (X. Zhang), wang-yuwang@mail.tsinghua.edu.cn (Y. Wang), wenjunzeng@eias.ac.cn (W. Zeng), nnzheng@mail.xjtu.edu.cn (N. Zheng).

¹ Co-first author

<https://doi.org/10.1016/j.neucom.2024.128200>

Received 7 November 2023; Received in revised form 5 July 2024; Accepted 14 July 2024

Available online 17 July 2024

0925-2312/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

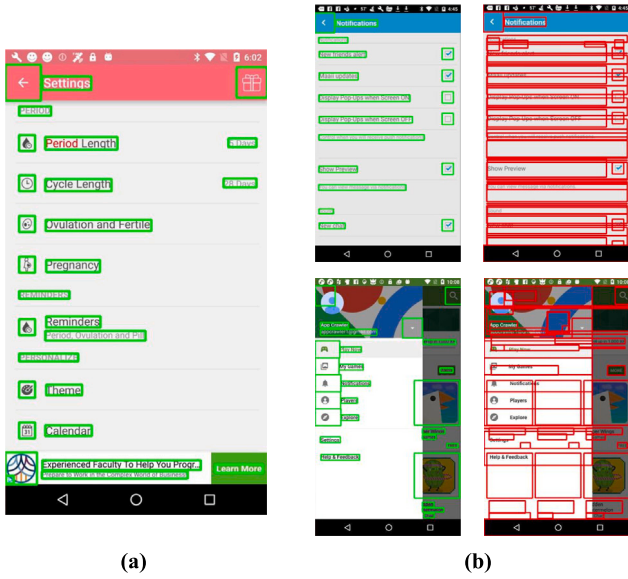


Fig. 1. (a) Visualization of our proposed *Pixel-Words* (labeled with green boxes). (b) Compared to leaf nodes in VH (labeled with red boxes), *Pixel-Words* are more clean and visually consistent across screenshots.

et al. [11] aim to generate the annotations of UI components by parsing the metadata with hand-crafted rules, but the quality is limited by the noisy metadata.

Our work is inspired by the modeling ideas in NLP. Each individual *Word* is an atomic component with the essential semantics, which is modeled as a token. All the tokens in a *Sentence* are fed into a Transformer to understand the whole *Sentence*. The key is to achieve the understanding of the whole sequence from the basic units. This successful modeling framework can be extended to understand UI. In analogy to NLP, the “word” of a screen should be: (1) atomic visual components carrying clear semantic meaning. (2) visually consistent across different UIs. A VH based work [7] takes the leaf nodes in metadata as the words for screen. However, as shown in Fig. 1(b), those leaf nodes are very noisy and some nodes contains only icons or texts, while some other leaf nodes contain both. The confusing semantics of the leaf nodes would hurt the understanding of individual components and the whole UI.

In this paper, we aim to achieve GUI understanding from pixels by extending the *Word/Sentence* concepts into the *Pixel-Word/Screen-Sentence* concepts, and propose a new architecture: from *Pixel-Words* to *Screen-Sentence* (PW2SS). Our PW2SS gets rid of the limitation of requiring metadata in inference and can be widely applied across different platforms, UI design tools and styles. The key is to design the *Pixel-Words* to have the property of *Words* in NLP. We define the *Pixel-Words* as atomic visual components of screens, which include *Text Pixel-Words* (text) extracted by OCR and *Graphic Pixel-Words* (icons and images) extracted by our Graphic Detector as shown in Fig. 1. The appearances of those *Pixel-Words* are clean and consistent across different screenshots. The benefits of our *Pixel-Words* definition are three-fold: (1) providing semantically effective tokens into the Transformer to understand the Screen-Sentences. (2) Enabling the OCR and Graphic Detector to extract *Pixel-Words* based on the consistent visual appearance of these components. (3) Making it possible to clean the noise in the metadata and get pseudo-labels of the *Pixel-Words* for the training of the detector. According to the above definition of *Pixel-Words*, we propose a heuristic method to generate pseudo label for *Pixel-Words* and build a *Pixel-Words* dataset named RICO-PW based on the published RICO dataset [12]. For *Screen-Sentence*, we leverage BERT [13] to design a Screen Transformer to model the relation of

Pixel-Words and train the Screen Transformer with masked *Pixel-Words* prediction.

To evaluate the effectiveness of our *Pixel-Words* and PW2SS, we conduct experiments on RICO-PW and a new GUI understanding dataset, P2S-UI, collected by ourselves. Our method achieves the best performance on the GUI understanding tasks including *Pixel-Words* extraction, relation prediction, clickability prediction, screen retrieval, and app type classification.

Our main contributions can be summarized as follows:

- We propose a pixel-based GUI understanding framework from pixels which is suitable for general applications across different platforms, UI design tools and styles.
- By defining the *Pixel-Words* as atomic text and graphic components, we make them visually consistent and semantically clear tokens for the Screen Transformer.
- We build a high-quality UI datasets RICO-PW with *Pixel-Words* annotations, which will be released to the public.

2. Related work

GUI Understanding is a challenging area which has attracted increasing attention recently due to the prevalence and importance of smart devices. One important task is extracting elements from screenshots and learning the representations of the components and the whole screen. The related works can be divided into two branches: purely vision-based methods which only take the screenshot as input and metadata-based methods which require the metadata of the screenshot as an extra input. The first branch suggests extracting UI components from screenshots using detection techniques. Some studies [10,14] utilize metadata from UI screenshots as annotations to train a UI component detector, whereas Zhang et al. [3] manually annotates UI components to achieve better detection results. However, all these methods are limited to component-level understanding and do not address the relationships between components and the entire screen. The second branch achieves screen level understanding with the help of metadata. Seq2act [1] retrieves elements in screen using structured query. ActionBert [7] and Screen2Vec [2] learn the embeddings of elements and screens. However, these methods depend on the metadata, which are not universally accessible and often suffer from noise.

Transformer-based representation learning aims to model the relations of tokens with Transformers and provide a global understanding of the whole input. The Transformer is proposed in [15] and BERT [13] is a well-accepted pretraining method for transformer. Recently, many works apply Transformers to visual language tasks [16–19]. Our work is different from these works in that they require extra text information, e.g. image caption, for each image, but our text information is extracted from screenshots by the OCR. The most related work to ours is LayoutLM [20,21] for form understanding. The main difference is that the graphic components like icons and images are more important in UI understanding, while LayoutLM mainly considers the text in the scanned document.

Mobile screen related applications focus on applying computer vision techniques in mobile UIs to solve particular tasks. Due to the ubiquity of the mobile phone, there are a lot of applications being explored. Reverse software engineering [4,5,22,23] aims to generate the code of UI from the corresponding screenshot. Clickability prediction [24–26] focuses on the prediction of UI element’s clickability. Design assistant [27–29] helps to design a GUI of apps. Assistants for the visually impaired [6] aims to generate descriptions of the elements. The previous works [30,31] detect display issue in mobile screens with computer vision technique. These papers mainly focus on the specific application scenario. Those tasks can be better accomplished

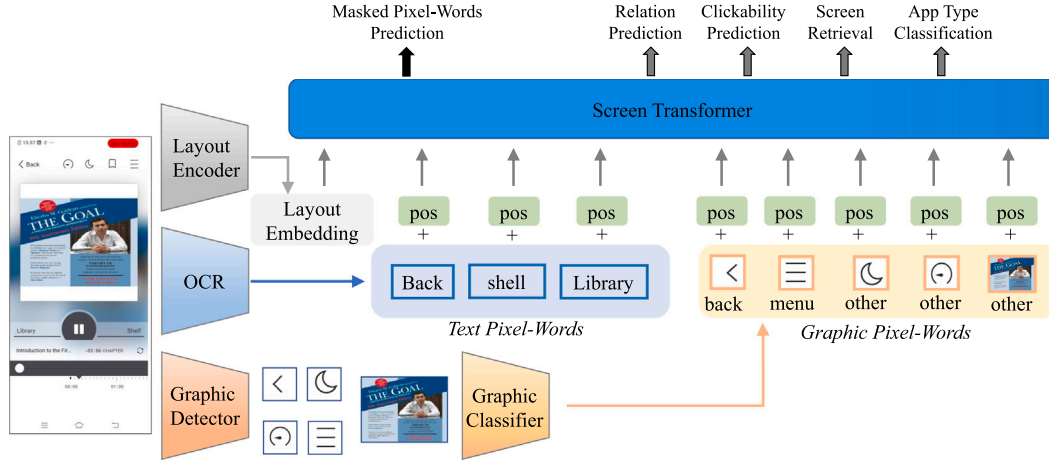


Fig. 2. Overview of our method. Given a screenshot, we first generate *Pixel-Words* with an OCR module to extract *Text Pixel-Words* and an Graphic Detector and Graphic Classifier to extract *Graphic Pixel-Words*. We also use a Layout Encoder to get a Layout Embedding proving a global layout information. All the *Pixel-Words* and the layout embedding are fed into the Screen Transformer to model the *Screen-Sentence*.

with a good understanding of the screens and can be regarded as the downstream tasks of our work.

Mobile screen related datasets are very important for the training of models of GUI understanding. For deep learning based methods, large scale datasets are necessary for reliable performance and generalizability. RICO [12] is one of the most important public datasets with both screenshots dataset with corresponding metadata in this area. However, it only contains some simple auto-generated labels without semantics. Liu et al. [11] aims to generate the semantic labels from the metadata, but it is still noisy and contains little semantic meaning about the screen. To address the issue of noisy labeling, some works, such as Li et al. [32], focus on solving this problem by providing auto-cleaning methods along with a newly human-labeled dataset. The key difference between our approach and these works is that we identify the phenomenon of visual ambiguity, which is caused not only by noisy labels but also by the unclear definition of atomic units of the screen (Detail in Section 5.2). Consequently, we introduce a new definition of the atomic unit, termed *Pixel-Words*, accompanied by a high-quality annotated dataset.

3. From Pixel-Words to Screen-Sentences

3.1. Overview of PW2SS

To understand the GUI from pixels, we extend concepts of *word/Sentence* in NLP to *Pixel-Words/Screen-Sentences*. Different from NLP, where the *Word* is already isolated, for GUI understanding, we need to first extract *Pixel-Words* from the given screenshot, then feed the *Pixel-Words* into our Screen Transformer as shown in Fig. 2. We first train the models to extract high-quality *Pixel-Words* and then train the Screen Transformer.

In analogy to *Words*, the *Pixel-Words* should be isolated visual components and carry elementary semantics. Screenshots are typically rendered from metadata, e.g. View Hierarchy (VH), describing the hierarchical structure of UI components in the screenshot. (Examples of VH data are shown in our Supplementary Material). One straightforward way to isolate “words” of a screen is to use the leaf nodes in VH as *Pixel Words*. However, as shown in Fig. 1, due to varieties of UI tools and coding styles, there are many invalid leaf nodes and the content in the leaf node varies across different screens. We define the *Pixel-Words* as the visual atomic components, which make up the screen. As Fig. 1(a) shows, our *Pixel-Words* include text components (e.g. text or blocks of texts) and graphic components (e.g. icons and

images), denoted as *Text Pixel-Words* and *Graphic Pixel-Words* respectively. Those *Pixel-Words* are both visually consistent and semantically carrying meaningful information.

Based on the *Pixel-Words*, the Screen Transformer is able to understand the whole screen. We refer to BERT [13] to add position embeddings to represent the locations of the *Pixel-Words* and use the masked *Pixel-Words* prediction task to pretrain the Screen Transformer. Furthermore, we design a layout embedding to provide global information of screen. Then the pretrained Screen Transformer can be finetuned to accomplish various downstream tasks like relation prediction, clickability prediction, screen retrieval and app type classification etc.

3.2. Pixel-Words extraction and understanding

We extract the *Pixel-Words* from a screenshot and transfer them into tokens based on different appearances of *Text Pixel-Words* and *Graphic Pixel-Words* as shown in Fig. 2. For the *Text Pixel-Words*, we use an off-the-shelf OCR tool² to extract the location and text content, then feed the text into Sentence-BERT [33] to get the representation of tokens. For the *Graphic Pixel-Words*, we design a Graphic Detector and Graphic Classifier to extract the location and semantics. The Graphic Detector detects the graphic elements, then we use a Classifier to identify the semantic meaning of the elements, and use the BERT embeddings of semantic labels as the tokens.

To train the Graphic Detector with less human labeling cost, we make use of VH in the RICO dataset [11,12]. The biggest challenge for getting supervision for UI components is that it is hard to filter out invalid nodes using visual features. However, our clear definition of *Graphic Pixel-Words* shows great advantages here. For *Graphic Pixel-Words*, we only focus on the atomic graphic components in the UI which have consistent appearance and can be easily distinguished from other complex UI components. The details are shown in the Supplementary Material.

According to our definition of *Pixel-Words*, a baseline method is to leverage the component annotations provided by Liu et al. [11] on RICO. We manually choose the components labeled with text-related categories as our *Text Pixel-Words*, and choose the components labeled with icon- or image-related categories as our *Graphic Pixel-Words*. However, the *Text Pixel-Words* collected in this way are still very noisy due to the low quality annotations of Liu et al. [11]. To obtain

² <https://developers.google.com/ml-kit/vision/text-recognition>.

Table 1

Comparison of the labels generated from the metadata using our method and Liu et al. [11] on 200 screenshots labeled by us. The “cleaning” means the operation of removing the screenshots with metadata.

Methods	Cleaning	Graphic <i>Pixel-Word</i>		Text <i>Pixel-Word</i>	
		Recall	Precision	Recall	Precision
Liu et al.	w/o	0.73	0.71	0.69	0.62
Liu et al.	w/	0.72	0.75	0.69	0.69
Ours	w/o	0.80	0.94	0.86	0.97
Ours	w/	0.85	0.95	0.95	0.97

a large number of *Pixel-Words* annotations with low cost of human effort, we carefully design a pipeline to extract annotation from VH. For text, we first extract nodes in VH with UI class names related to “text”, then we use the OCR tool to localize text inside these nodes. For graphics, we generate candidates according to location of texts and nodes in VH, then we use a binary classifier to identify each candidate is a graphic or not. We collect 1239 candidate patches from screenshots and manually label them as positive or negative samples. The binary classifier is trained on this dataset and demonstrates high accuracy performance. We further propose a “cleaning” operation to filter out invalid screenshots where there is an obvious mismatch between the number of text boxes extracted using OCR or metadata.

To verify the effectiveness of the generated labels using our method, we manually label the *Pixel-Words* in 200 screenshots randomly sampled from RICO and use them as ground truth. As Table 1 shows, our generated labels for *Pixel-Words* achieve higher Recall and Precision than the leaf nodes of VH.

For further understanding the extracted graphics, we use a Graphic Classifier to recognize the semantic meaning of each graphics, as shown in Fig. 2. To train the Graphic Classifier, we build a high quality graphic dataset, named RICO-ICON, by refining category annotations from RICO. Based on the observation that there exists a serious long-tail problem of icon categories in the original RICO dataset, we only collect the most important categories in RICO-ICON. We use the clicking frequency as a metric to measure the importance of graphic categories and select 31 categories with the highest clicking frequency as the graphic categories in RICO-ICON. The remaining categories of graphic are assigned an “other” category label.

3.3. Screen-Sentence understanding

To accomplish tasks requiring semantic understanding of the whole screen, we take the *Pixel-Words* from the previous stage as tokens and feed them into the Screen Transformer to model their relation and form a *Screen-Sentence*. We design our Screen Transformer as a 6-layer transformer architecture referring to BERT [13]. As Fig. 2 shows, the input of the transformer consists of three parts: *Pixel-Words* embeddings, corresponding 2-D position embeddings and a layout embedding. *Pixel-Words* embeddings represent the semantics of these atomic visual components. The embeddings of *Text Pixel-Words* and *Graphic Pixel-Words* are processed by a linear layer to ensure they have the same size. Position embeddings are added to each corresponding *Pixel-Words* to provide spatial position. Following the same setting as LayoutLM [20], we represent the position and size of *Pixel-Words* in screenshot as $(x_{min}, y_{min}, x_{max}, y_{max}, w, h)$, where w and h denote the width and height of the *Pixel-Words* respectively. A layout embedding is designed to provide the global layout information of the components. We follow Liu et al. [11] to generate layout representation using the bounding boxes of our *Pixel-Words*.

The Screen Transformer is pretrained with a self-supervised task: masked *Pixel-Words* prediction. We select four downstream tasks covering the understanding from *Pixel-Words* level to *Sentence-Screen* level. Those tasks can be well used to evaluate the understanding performance and are practically important.

Pretraining Task: Mask *Pixel-Words* prediction Inspired by BERT [13], where the input tokens are randomly masked and predicted from the remaining tokens, we design the masked *Pixel-Words* prediction task to regress the embedding of the masked *Pixel-Words*. The purpose is to force the transformer to learn the dependencies among *Pixel-Words* in the screen. More specifically, we randomly mask 15% of the total *Pixel-Words* fed into the Screen Transformer, and take the objective function as ℓ_2 norm between the masked *Pixel-Words* and predicted *Pixel-Words*.

Downstream task #1: Clickability Prediction We define the clickability of a *Pixel-Word* as whether the screen changes after clicking on it or not. Following this definition, the clickability of the *Pixel-Words* is annotated by actually clicking and observing the screen’s change. The task is to predict whether the *Pixel-Word* is clickable or not. We use a classifier (a 3-layer MLP) to determine the clickability of the *Pixel-Word* from the embedding output by the Screen Transformer.

Downstream task #2: Relation Prediction The relation prediction task is to predict the pairwise relation between *Pixel-Word* pairs. We establish a relationship between two *Pixel-Words* if they serve the same specific function. For instance, an app name is related to the app icon because both aid in guiding users to open the app. A key criterion for determining whether two clickable *Pixel-Words* are related is (1) the identical change in the screen after clicking on them or (2) one *pixel-words* is described the function of the other *pixel-words*. For example, as shown in Fig. 8(b), the WiFi icon and the texts “WiFi” nearby have the same semantic meaning and they should have close relation. We first sum the embeddings of two *Pixel-Words* output from the Screen Transformer and feed the result into a 3-layer multi-layer perceptron (MLP) to predict the relation of the two *Pixel-Words*. The objective function is the cross entropy between the predicted relation and the ground truth.

Downstream task #3: Screen Retrieval In analogy to sentence retrieval in NLP, screen retrieval task aims to find the screenshots that are the most similar to the query screenshot in terms of both the semantics of the content and the layout of the structure. To accomplish screen retrieval, a good understanding of the whole screen is required. To obtain a representation for a given screenshot, we apply the max pooling on all the outputs of the Screen Transformer. Then we find the closest screenshots for the query screenshot using the cosine similarity of the representations of the screenshots.

Downstream task #4: App Classification The app type classification task is to recognize the app type for each screenshot. We also use a maxpooling operation to aggregate all the output embedding from Screen Transformer to get the representation of the given screen. Then we feed the representation into a classifier (a 3-layer MLP) to predict the app type of the screen.

4. Dataset construction

4.1. RICO

RICO [11,12] is a public dataset on mobile UI. There are a total of 66,261 screenshots with corresponding metadata. The dataset is first proposed by Deka et al. [12] with screenshots and VH files only and Liu et al. [11] add semantic labels for every node of VH. RICO-PW is a dataset built by us for

4.1.1. Details of *Pixel-Words* annotation generation

To solve the invalid node problem in VH, motivated by “proposal-classification” [34–36] style detection method. We try to extract proposals from the VH then use a classifier to identify each proposal is a graphic or not.

To train such a binary classifier, we collect 1273 patches of VH nodes from the original screenshots of RICO. 694 of them are labeled

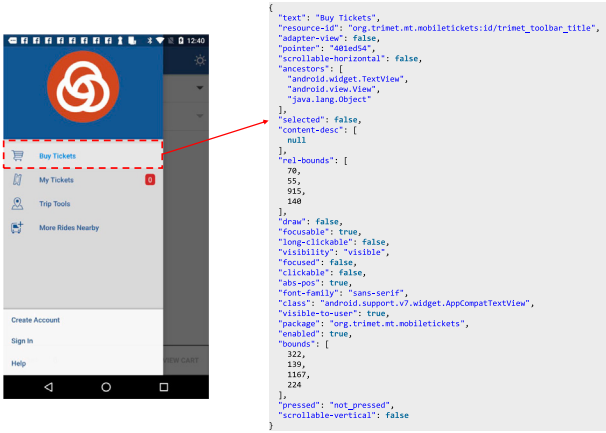


Fig. 3. A typical node data in View Hierarchy (VH) from RICO dataset.

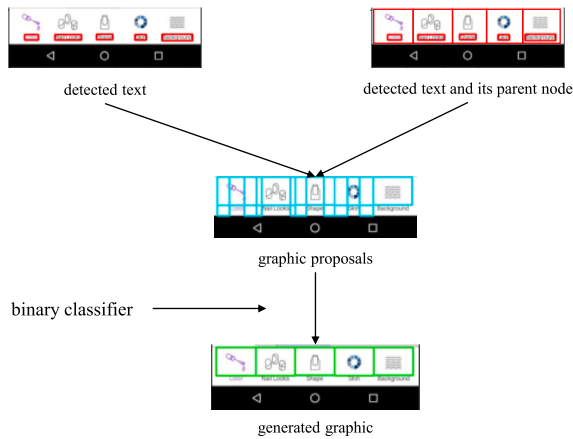


Fig. 4. The visualization of progress to generate missing graphic in View Hierarchy (VH).

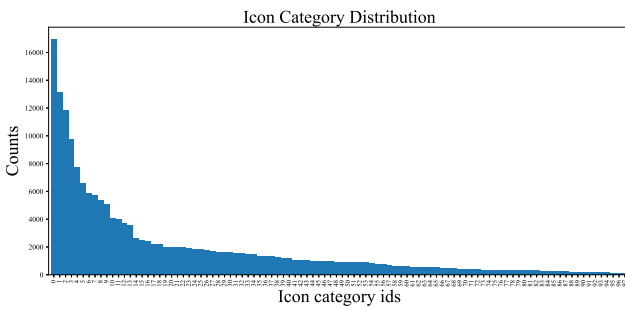


Fig. 5. Long tail distribution of ICON category.

as positive samples and the rest 539 patches are labeled as negative samples. After training, our classifier achieves 0.95 accuracy.

However, there are two challenges here. First, there are a huge number of nodes in VH, if we take every node as a proposal it will be pretty inefficient and increase the number of failure cases for our classification. Second, about 28% of the graphics do not have corresponding nodes in VH, so it means the upper bound of recall will be very limited. To tackle the first challenge, we create a candidate set of node class names that covers all graphics-related categories. When we sample nodes from VH, only the node whose class name can be found in the candidate set will be considered as possible proposals for graphics. For the second problem, we observed that the missing graphic nodes often associate with texts as Fig. 3 shows. Therefore we generate new

proposals according to the texts and their parent nodes. Specifically, for a text and its parent node, we select the regions between the text and its parent node's top/bottom/left/right boundaries as proposals respectively, as shown in Fig. 4. The methods solving the first and second challenge helps to improve the precision and recall respectively.

```

1 # nodes: List[Dict], all nodes data in a VH file
2 # clsname_candidates: Set(str), a set which
  includes possible graphic class in VH.
3 # text_bboxes: List[List[int]], the list of text's
  bounding boxes in current screenshots.
4
5 proposals = []
6 # step1: filter out unrelated nodes.
7 for node in nodes:
8     if node["class"] in clsname_candidates or node[
9         "ancestors"] in clsname_candidates:
10         proposals.append(node["bound"])
11
12 # step2: add spaced_region between text and its
  parent node.
13 for text in text_bboxes:
14     parent = get_parent_node_bbox(text)
15     proposals += generate_spaced_region(parent,
16         text)
17
18 # step3: use the binary classifier to evaluate the
  proposals.
19 graphics_bboxes = []
20 for proposal in proposals:
21     if evaluate(proposal) > score_thres:
22         graphics_bboxes.append(proposal)
23 return graphics_bboxes

```

Listing 1: pseudo code for *Graphic Pixel-Words* Annotation Generation

4.1.2. Generation of RICO-ICON

To train the Graphic Classifier, we build a high quality graphic dataset, named RICO-ICON, by refining category annotations from RICO. Based on the observation that there exists a serious long-tail problem of icon categories in the original RICO dataset (Detail in Fig. 5), we only collect the most important categories in RICO-ICON. We use the clicking frequency as a metric to measure the importance of graphic categories and select 31 categories with the highest clicking frequency as the graphic categories in RICO-ICON. The remaining categories of graphic are assigned an “other” category label. RICO-ICON dataset is consisting of 120,067 icons cropped from the RICO dataset. The icons are divided into 32 categories.

4.2. P2S-UI

P2S-UI is a dataset collected by ourselves. This dataset contains 27,077 mobile UI images. There also are the annotations for relation, clickability and app type for each screenshot. The detail information for the annotation is given below:

Clickability and Relation The clickability and relations are labeled manually according to the definitions provided in Section 3.3. The detailed process is outlined below:

- We first trained a *Pixel-Words* detector using the RICO-PW dataset (Detailed in Section 5.2). This detector is used to generate pre-labels in the P2S-UI dataset. The pre-labeling primarily includes bounding box annotations and the classification of elements as either text or graphics.
- Each image in the P2S-UI dataset, along with its pre-labels, is assigned to two labelers. The labelers are required to: (1) verify the accuracy of the bounding box annotations and the classification of text and graphics, and (2) provide clickability and relation labels for each *Pixel-Word*.

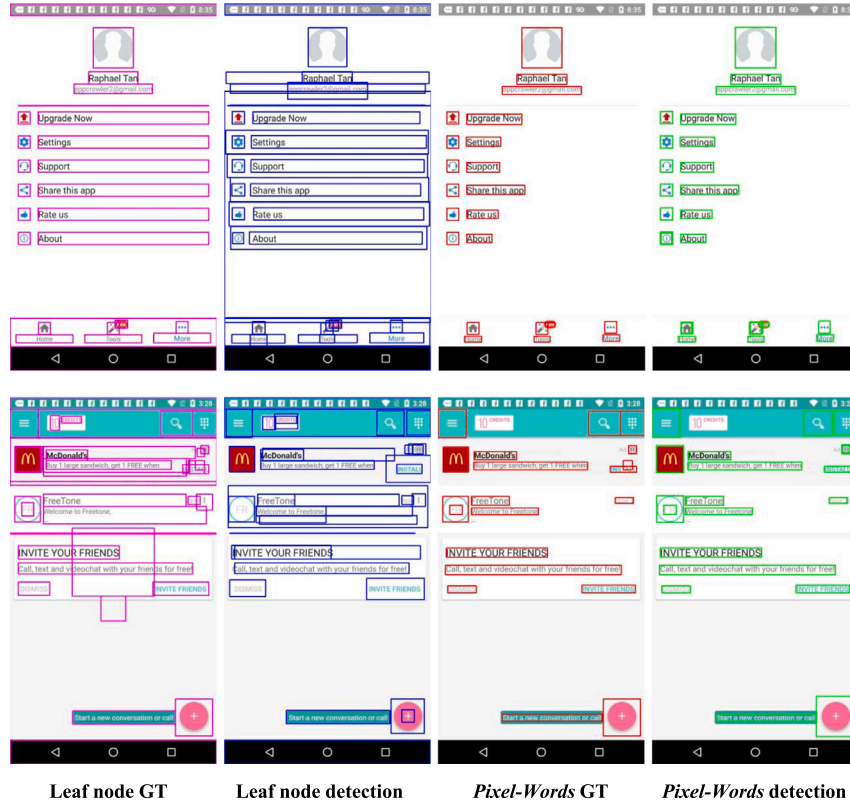


Fig. 6. Visualization of leaf nodes GT (magenta), leaf nodes detection results (blue), *Pixel-Words* ground-truth (red) and *Pixel-Words* detection results (green).

- We run an algorithm to check the consistency of the labels provided by the two labelers. If the labels are inconsistent, the discrepancies are sent to a third labeler, and the results from the third labeler are used as the final labels.

App Types The information about the app type is gathered through the meta-information of the app from the app store. Our dataset comprises 26 classes of apps. The distribution of these 26 classes and Visualization of some classes of the app is provided Appendix.

5. Experiment

5.1. Experimental setup

Baselines **Leaf nodes** are used as the input of Transformer in the previous work [7], which can be regarded as the baseline of *Pixel-Words*. When extracting the leaf nodes, we use heuristic rules (e.g. spatial size and ratio of the nodes) to filter out some invalid leaf nodes. **W/o pretraining** is the baseline that the models trained without the RICO-PW dataset to study the impact of pretraining on the RICO-PW dataset. **W/o Screen Transformer** is the baseline to study the impact of our proposed Screen Transformer.

Metrics To evaluate the performance of *Pixel-Words* detection, we use Intersection over Union (IoU)-based detection metrics, which includes Average Recall (AR) of COCO-style [37], Average Precision (AP) of COCO-style, Average Precision with 0.50 IoU threshold (AP50), and Average Precision with 0.75 IoU threshold (AP75). We also use “Center” metric referring to Zhang et al. [3]. Given a predicted bounding box, the criteria of true or false is whether the center of the bounding box is inside the ground-truth box or not. For graphic classification, clickability prediction and relation prediction tasks, we use top1 accuracy as our metric.

Table 2

Comparison of the detection results between leaf node and *Pixel-Words* on RICO-PW.

Method	Metric			
	AR	AP	AP50	AP75
Leaf Nodes	0.665	0.575	0.720	0.611
<i>Pixel-Words</i>	0.780	0.636	0.835	0.746

Table 3

Comparison of different detectors' performance on RICO-PW validation and test splits. AR is short for Average Recall. RTN-S, ATSS-S, FA-S denotes RetinaNet, ATSS, FreeAnchor with ResNet50 [38] as the backbone respectively. FA-D denotes FreeAnchor with ResNeXt101 [39] as backbone.

Models	Test			
	AR	AP	AP50	AP75
RTN-S	0.720	0.609	0.748	0.654
ATSS-S	0.718	0.587	0.778	0.628
FA-S	0.750	0.645	0.749	0.693
FA-D	0.792	0.676	0.837	0.722

Dataset Split The dataset is splitted for training, validation and test according to the apps, as each app typically comprises several comparable screenshots. The division of the dataset follows a 7:1:2 ratio for the training set, validation set, and test set, respectively.

5.2. *Pixel-Words* evaluation

In this section, we evaluate the extracting of *Pixel-Words* from screenshots on RICO-PW and P2S-UI. We take 67.5%, 7.5% and 25% of screenshots for graphic detector's training, validation and testing respectively. During pretraining, we combine the training set and test set to obtain a better pretrained detector.

Study of Different Detection Models We select several typical one-stage detectors (for efficient inference) equipped with FPN to study

Table 4

The detecting results of *Graphic Pixel-Words* on P2S-UI with ImageNet or RICO-PW pretraining. IoU and Center metrics are used for evaluation.

Setting	IoU				Center	
	AR	AP	AP50	AP75	Recall	AP
ImageNet pretraining	0.693	0.601	0.899	0.676	0.963	0.922
RICO-PW pretraining	0.714	0.634	0.916	0.741	0.961	0.938

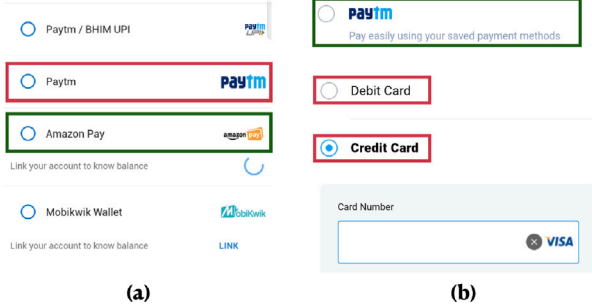


Fig. 7. Visual depiction showcasing the ambiguity of the Leaf Node: **Green Box:** Ambiguity stemming from the unclear definition of labeling criterion. **Red Box:** Ambiguity arising from the variation in the screenshot.

the impact of different detectors on RICO-PW dataset. RetinaNet [40] is a generic one-stage detector, which is equipped with focal loss to solve the imbalance between positive and negative samples and assign labels on the basis of hand-crafted IoU threshold. ATSS [41] proposes a new mechanism which can adaptively adjust the IoU threshold to assign label. FreeAnchor [42] treats the label assignment problem as a maximum estimation problem, which achieves the “learning to assign” during the training progress. Table 3 shows results from above detectors. FreeAnchor obtains the highest AR and AP score, which means it performs well on both recognition and localization. One possible reason is that different from natural images, there are usually a lot of blank or texture-less backgrounds in the target bounding boxes in screenshots. FreeAnchor can better handle this problem and find the informative anchors to represent the detection target. Therefore We use the FreeAnchor detector in following experiments.

Pixel-Words v.s. Leaf Nodes We study which type of “word” of screenshots (*Pixel-Words* or leaf nodes) is easier to be extracted. The *Pixel-Words* and leaf nodes are learnt with the supervision of the cleaned leaf nodes and *Pixel-Words* annotations respectively on RICO-PW. Visualized comparison are shown in Fig. 6. Compared to leaf nodes, as Table 2 shows, our *Pixel-Words* gain 17.29%, 10.60%, 15.87%, 22.09% on the metrics of AR, AP, AP50, AP75 respectively. The result supports our assumption that *Pixel-Words* are easier to be extracted from pixels than leaf nodes and better fit our PW2SS framework.

Visual ambiguity One of the primary attributes of *Pixel-Words* is their ability to alleviate visual ambiguity. We define visual ambiguity as the variability of labeling criteria across different elements, which can arise from unclear definitions or variations in screenshots. For instance, in Fig. 7, the text is not enclosed within the green box in (a) but is in (b), illustrating inconsistency in labeling criteria. Additionally, variations in screenshots, such as the red box in (a) containing a blue circle, text, and an icon, while in (b) it contains only the blue circle and text, contribute to visual ambiguity. This ambiguity can hinder detection; for instance, when elements containing *a, b, c* and *a, b* are both valid, detectors may struggle to differentiate, leading to predictions of all possible elements. This issue is evident in the leaf node detection case in Fig. 6. By defining *Pixel-Words*, we mitigate ambiguity, enabling clearer detection, as demonstrated in Fig. 6 with *Pixel-Words* detection.

Study of pretraining on RICO-PW We take our RICO-PW dataset as a pretraining dataset and finetune the detector on the human labeled

Table 5

Top1 accuracy of our Graphic classifier trained on RICO-ICON.

Category	Top1 acc	Category	Top1 acc
add	0.944	favorite	0.962
arrow_backward	0.995	filter	0.880
arrow_downward	0.981	gallery	0.930
arrow_forward	0.977	location	0.940
arrow_upward	0.876	menu	0.985
avatar	0.904	microphone	0.945
calendar	0.897	more	0.978
call	0.930	other	0.929
camera	0.900	pause	0.855
cart	0.947	play	0.974
chat	0.937	question_mark	0.968
check	0.966	refresh	0.965
close	0.951	search	0.941
delete	0.896	send	0.941
download	0.998	settings	0.905
edit	0.942	share	0.990
Average	0.957		

P2S-UI dataset. Here we take the common ImageNet [43] pretraining as baseline. Table 4 shows our results. Compared to ImageNet pretraining, our RICO-PW pretraining gains 3.03%, 5.49%, 1.89%, 9.62% on AR, AP, AP50, AP75 respectively. The biggest improvement comes from AP75, which means the pretraining helps to localize graphics more precisely.

Study of Graphic Classifier The graphics play an important role in GUI, which can provide rich semantic meaning in limited pixels. We try to recognize the graphics as fine-grained categories to represent the semantic meanings. As mentioned in Section 3.2, we reorganize the categories from original RICO dataset and train our graphic classifier on that. Here we use MobilenetV2 [44] as our Graphic Classifier. We report our category-level top1 accuracy in Table 5. The results show our Graphic Classifier can get rich information from our dataset. We achieve 0.975 average top1 accuracy of 32 categories. The reason Table 5 does not include other types of UI elements, such as radio buttons or checkboxes, is: (1) the long-tailed nature of graphic UI components (Detail in Section 4.1.2) and we prioritized the selection based on the frequency of user interactions, (2) In our focusing the target, these elements do not have significant role; *The better Graphic Classifier is possible in the future by giving more detail class labels for the graphic fall in “other” class.*

5.3. PW2SS evaluation

We first pretrain the Screen Transformer with the masked *Pixel-Words* prediction task, then we evaluate the effectiveness of PW2SS on *Pixel-Word* level tasks (relation prediction and clickability prediction) and *Screen-Sentence* level tasks (screen and app classification). We leverage the FA-D detection to extract the *Graphic Pixel-Words* as it achieves best performance.

Pretraining on RICO-PW Before we train our Screen Transformer in downstream tasks, we first pretrain it on the RICO-PW dataset. The goal of pretraining is to force the Screen Transformer to model relations between *Pixel-Words* using the task of Mask *Pixel-Words* Model. For each screenshot, 15% of *Pixel-Words* are randomly masked. We pretrain the model for 50 epochs, and using 5 epoch for warming up. We use the AdamW optimizer with learning rate $r = 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-6}$ and batch size of 64 for training.

Clickability Prediction In this task, we add a classifier head on the output feature of the Screen Transformer to predict whether a *Pixel-Word* is clickable or not. Table 6 shows that the with the Screen Transformer, we can achieve better clickability results using the context of *Pixel-Words*. Besides, the pretraining on RICO-PW helps to further improve the accuracy, which indicates that the masked *Pixel-Words* task



Fig. 8. Visualization of relations prediction results of PW2SS on P2S-UI. A line is shown to connect two *Pixel-Words* with relation. The predicted results is labeled with blue lines and ground truth is labeled with red lines.

Table 6
Results of Clickability Prediction (CP) and Relation Prediction (RP) on P2S-UI.

Setting	CP	RP
w/o Screen Transformer	0.871	0.906
w/ Screen Transformer (w/o pretraining)	0.896	0.947
w/ Screen Transformer (w/ pretraining)	0.910	0.965

Table 7
App type classification results on P2S-UI. We study the impact of different inputs of Screen Transformer: *Pixel-Words* v.s. Leaf Nodes, and the impact of Screen Transformer.

Setting	Model	Accuracy
Leaf Node	w/o Screen Transformer	0.914
	w/ Screen Transformer	0.915
<i>Pixel-Words</i>	w/o Screen Transformer	0.948
	w/ Screen Transformer	0.955

is effective to learn a better representation by modeling the relation between *Pixel-Words*.

Relation Prediction Relation prediction task is designed to evaluate the performance of our Screen Transformer on the tasks about *Pixel-Words* pairs. Examples of the relation are shown in Fig. 8. As Table 6 shows, similar to the clickability prediction task, both the Screen Transformer and pretraining are effective for these tasks. An interesting observation is that, compared to the clickability prediction task, the gain caused by Screen Transformer in relation prediction task is larger. The reason is that relation prediction task relies more on the context of *Pixel-Words*, where the Screen Transformer can provide more useful information.

App Type Classification All the screenshots in P2S-UI dataset are divided into the 26 categories according to the app type (e.g. SHOP, SOCIAL). We learn to predict the app type for each *Screen-Sentence*. As Table 7 shows, our *Pixel-Words* outperform leaf nodes with more than 3% gain, which verifies the effectiveness of our defined *Pixel-Words* in screen-level understanding. When comparing with versions lacking Screen Transformer, we can confirm two points: (1) the significance of enhancing the representation of each component with its context, and (2) the efficacy of the Screen Transformer framework proposed.

Screen Retrieval Screen Retrieval is a well-applied task to evaluate the models' ability to learn the representation of the whole screen. Given the query screen, we will retrieval screenshots which is similar to the query screen. The query image is sampled from the training set, and the related image is retrieved from the test set. We compare our model to the method [11], which uses a autoencoder of the screen layout to retrieval screens. As Fig. 9 shows, the screen embedding generated by Screen Transformer is more suitable to retrieve the screens having similar semantic meaning as well as spatial layout. To further assess the

Table 8
Numerical assessment of the retrieval performance. The evaluation is based on the proportion of retrieved images that belong to the same app type class as the query image.

Methods	Rank 1	Rank 2	Rank 3
Liu et al.	0.998	0.772	0.574
Ours	0.999	0.816	0.635

efficacy of the retrieval method, we provide a quantitative evaluation of its performance in Table 8. This evaluation focuses on determining the quality of the retrieval results by assessing whether the retrieved screens belong to the same app type.

5.4. Compared with prior works

We categorized the prior works aiming to learn general representation for screen tasks into two groups: two-stage methods and one-stage methods. Two-stage methods involve initially extracting elements, such as *Pixel-Words*, from the screen, followed by extracting and refining their representations. On the other hand, one-stage methods combine element extraction and representation learning into a single step. Our work falls under the two-stage method category. Furthermore, there are also works distinct from those mentioned earlier, which focus on addressing a singular task rather than aiming for general representation learning. These are referred to as task-specific methods. In the following sections, we will compare our approach with prior works from both of these categories.

5.4.1. Compared with two-stage methods

We provide a numeric comparison with prior works, namely Screen Parser [45], Screen2Vec [2] and UIBert [46], in Table 8. We have reimplemented the prior methods using our dataset, with reproduction details provided in Appendix. Our method demonstrates superior performance compared to the prior works in Table 9.

Semantic informations A fundamental aspect of our work is the integration of rich semantic information within *Pixel-Words*. For textual *Pixel-Words*, we utilize pretrained SentenceBert to extract their semantic meaning, while for graphical *Pixel-Words*, we employ the Graphic Classifier to discern their functional semantics. Previous studies either predominantly concentrate on the layout information provided by ViewHierarchy, which is frequently characterized by its coarse and ambiguous nature, as outlined in Section 5.2 [2,46], or overlook the semantic information [45]. In contrast, our approach advances semantic comprehension by defining atomic semantic units, and enriches the semantic information using pretrained Graphic classifiers and SentenceBert.

Context information The concept of *Pixel-Words* solely encapsulates unit semantics. We utilize Transformer to augment the representation

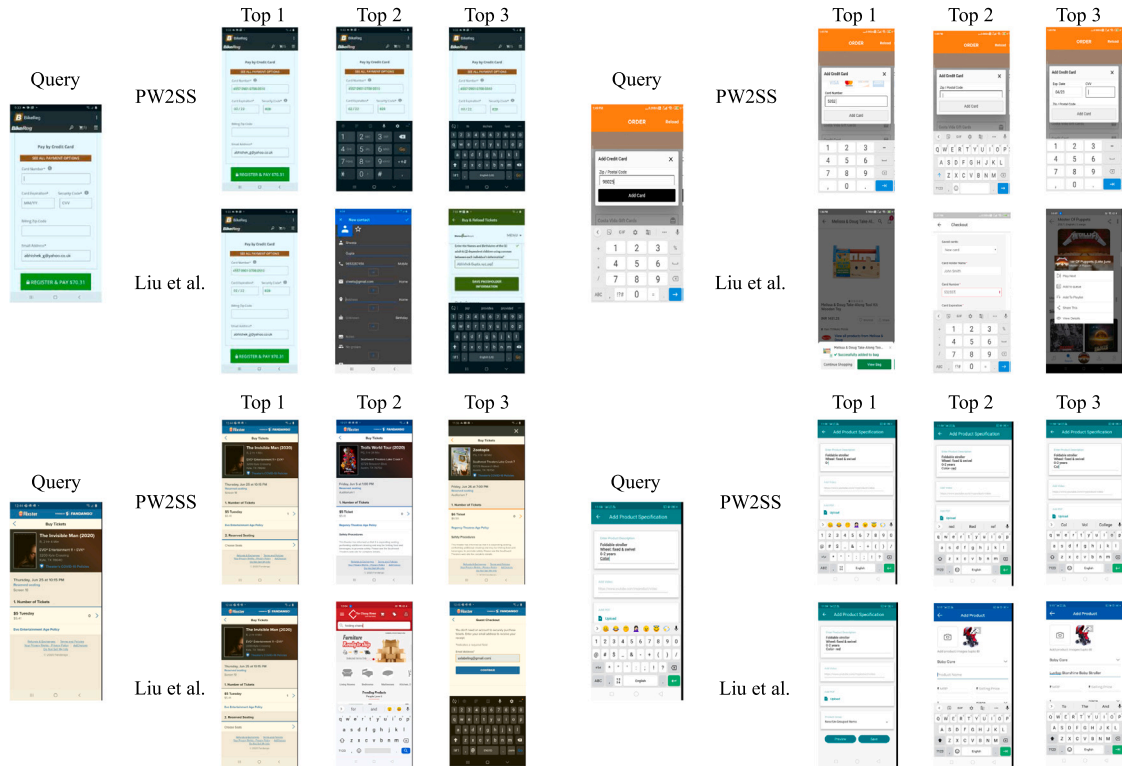


Fig. 9. Comparison of the retrieval results of our PW2SS and Liu et al. [11] on P2S-UI. The retrieved screenshots of PW2SS are more similar to the query screenshots in terms of both semantics and layout.

of *Pixel-Words* by incorporating contextual information. Pretraining is instrumental in facilitating communication of information among *Pixel-Words*. Some advantages of our approach stem from the Transformer's superior representational capacity compared to traditional RNN structures [2,45].

5.4.2. Compared with one-stage methods

There exists another category of research that employs one-stage methods [47], where elements detection and other downstream tasks are combined. One of the advantages of such methods is their structural parsimony and faster inference. However, they encounter challenges in extracting rich semantic information. In our framework, we utilize pretrained OCR tools and SentenceBert to extract semantic meaning from textual *Pixel-Words*, and we leverage object detection and classifiers to discern semantic meaning from graphical *Pixel-Words*. Combining all these components into a one-stage method would preclude us from leveraging the trained OCR tool and SentenceBert separately, as well as necessitate training the Graphic Classifier on the same dataset. This would entail labeling all this information in our screen dataset, significantly increasing labeling costs compared to our current framework.

5.4.3. Compared with task-specific methods

In contrast to our work and the aforementioned studies that aim to achieve a generalized representation for screens, some research solely focuses on specific tasks. For example, Sweargin and Li [25], Wu et al. [26] and Schoop et al. [24] concentrate on predicting clickability. These methods prove advantageous when computational resources are limited and only one task is pertinent. However, developing separate models for each task becomes inefficient when multiple tasks require attention, as it hinders the sharing of knowledge across tasks. Additionally, these methods [24–26] may not fully leverage contextual and semantic information from the screen. They treat clickability prediction akin to image classification [24,25] or object detection [26], where Sweargin and Li [25] integrates text embeddings but with

Table 9

In comparing with earlier research, including ScreenParsing [45], Screen2Vec [2] and UIBert [46], the VH indicator denotes whether the methodology extracts elements from the ViewHierarchy or relies on an object detector working at the pixel level to identify such elements. Our method outperforms previous works because (1) The *Pixel-Words* can convey clear and rich semantic information and (2) the Transformer structure and Mask *Pixel-Words* Prediction excel in extracting the context information. Note that we do not compare the clickability with the methods using VH because we believe that comparing the clickability prediction accuracy of *Pixel-Words* with the clickability accuracy of elements in the VH is not meaningful. The same for relation prediction. A detailed analysis of the advantages of our method is presented below.

Methods	VH	CP	RP	ATC
ScreenParsing		0.869	0.896	0.891
Screen2Vec	✓	/	/	0.854
UIBert	✓	/	/	0.923
Ours		0.910	0.965	0.955

diminished semantic fidelity due to relying on Bag of Words (BoW) embeddings and word counts, neglecting deeper sentence and graphical meanings. The primary limitation in utilizing contextual and semantic cues stems from the inefficiency of extracting such information for a single task. Handling multiple tasks simultaneously can render this extraction more feasible.

6. Discussion

What is the task understanding from the Pixel? The objective of GUI understanding from pixels is to achieve comprehension of graphical user interfaces using only the screenshot as the input source. This differs from other approaches that utilize ViewHierarchy information as an auxiliary input. The motivation for excluding ViewHierarchy is twofold: it is not consistently available, and its content can be imprecise. In this work, we introduce the concept of *Pixel-Words* as the fundamental units of semantic meaning. Leveraging these unit

semantics, we employ the Transformer model to decipher their relationships. With this foundation, we address both element-wise tasks such as clickability prediction and relation prediction, as well as screen-wise tasks including screen retrieval and app type classification. This innovative method paves the path for tackling more intricate tasks, like question answering and task automation, directly from pixel data.

7. Conclusion

In this work, we propose a pixel-based GUI understanding framework, PW2SS, which is suitable for general applications across different platforms, UI design tools and styles. We extend the *Word/Sentence* concepts into the *Pixel-Word/Screen-Sentence* concepts in the GUI understanding area. *Pixel-Words* are defined as atomic components with essential and clear semantics. Based on the definition of the *Pixel-Words*, we can use OCR and graphic detector to extract the *Pixel-Words* from the screenshots. Then a Screen Transformer is proposed to model the relation between *Pixel-Words*. The effectiveness of PW2SS is verified in tasks including *Pixel-Words* extraction, relation prediction, clickability prediction, screen retrieval, and app type classification. We can also extend our work to understanding screenshot sequences of user accomplishing various tasks by aggregating *Screen-Sentences* into *Task-Paragraphs*.

CRedit authorship contribution statement

Jingwen Fu: Writing – original draft, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Xiaoyi Zhang:** Writing – original draft, Methodology, Investigation, Data curation, Conceptualization. **Yuwang Wang:** Writing – review & editing, Supervision, Methodology, Conceptualization. **Wenjun Zeng:** Writing – review & editing, Supervision, Conceptualization. **Nanning Zheng:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The work was supported in part with the National Natural Science Foundation of China (Grant No. 62088102).

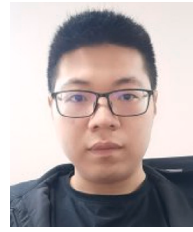
Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.neucom.2024.128200>.

References

- [1] Y. Li, J. He, X. Zhou, Y. Zhang, J. Baldridge, Mapping natural language instructions to mobile UI action sequences, in: Annual Conference of the Association for Computational Linguistics, ACL 2020, 2020.
- [2] T.J.-J. Li, L. Popowski, T.M. Mitchell, B.A. Myers, Screen2Vec: Semantic embedding of GUI screens and GUI components, 2021, arXiv preprint arXiv:2101.11103.
- [3] X. Zhang, L. de Greef, A. Swearngin, S. White, K. Murray, L. Yu, Q. Shan, J. Nichols, J. Wu, C. Fleizach, et al., Screen recognition: Creating accessibility metadata for mobile applications from pixels, in: The 2021 ACM Conference on Human Factors in Computing Systems, CHI 2021, 2021.
- [4] T. Beltramelli, Pix2code: Generating code from a graphical user interface screenshot, in: Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems, EICS 2018, 2018.
- [5] T.A. Nguyen, C. Csallner, Reverse engineering mobile application user interfaces with remai (t), in: 2015 30th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2015.
- [6] J. Chen, C. Chen, Z. Xing, X. Xu, L. Zhu, G. Li, J. Wang, Unblind your apps: Predicting natural-language labels for mobile gui components by deep learning, in: 2020 IEEE/ACM 42nd International Conference on Software Engineering, ICSE, IEEE, 2020.
- [7] Z. He, S. Sunkara, X. Zang, Y. Xu, L. Liu, N. Wichers, G. Schubiner, R. Lee, J. Chen, ActionBert: Leveraging user actions for semantic understanding of user interfaces, in: AAAI Conference on Artificial Intelligence, AAAI 2021, 2021.
- [8] X. Zhang, A.S. Ross, A. Caspi, J. Fogarty, J.O. Wobbrock, Interaction proxies for runtime repair and enhancement of mobile application accessibility, in: Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems, CHI 2017, 2017.
- [9] X. Zhang, A.S. Ross, J. Fogarty, Robust annotation of mobile application interfaces in methods for accessibility repair and enhancement, in: Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology, UIST 2018, 2018.
- [10] J. Chen, M. Xie, Z. Xing, C. Chen, X. Xu, L. Zhu, G. Li, Object detection for graphical user interface: old fashioned or deep learning or a combination? in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2020.
- [11] T.F. Liu, M. Craft, J. Situ, E. Yumer, R. Mech, R. Kumar, Learning design semantics for mobile apps, in: Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology, UIST 2018, 2018.
- [12] B. Deka, Z. Huang, C. Franzen, J. Hibschan, D. Afergan, Y. Li, J. Nichols, R. Kumar, Rico: A mobile app dataset for building data-driven design applications, in: Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology, UIST 2017, 2017.
- [13] J. Devlin, M. Chang, K. Lee, K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT.
- [14] X. Sun, T. Li, J. Xu, UI components recognition system based on image understanding, in: 2020 IEEE 20th International Conference on Software Quality, Reliability and Security Companion, QRS-C 2020, IEEE, 2020.
- [15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, NeurIPS 2017, 2017.
- [16] Z. Huang, Z. Zeng, B. Liu, D. Fu, J. Fu, Pixel-BERT: Aligning image pixels with text by deep multi-modal transformers, 2020, CoRR abs/2004.00849. arXiv:2004.00849.
- [17] J. Lu, D. Batra, D. Parikh, S. Lee, ViLBERT: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks, in: H.M. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E.B. Fox, R. Garnett (Eds.), Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada, 2019.
- [18] G. Li, N. Duan, Y. Fang, M. Gong, D. Jiang, Unicoder-vl: A universal encoder for vision and language by cross-modal pre-training, in: Proceedings of the AAAI Conference on Artificial Intelligence, AAAI2020, Vol. 34, 2020.
- [19] W. Su, X. Zhu, Y. Cao, B. Li, L. Lu, F. Wei, J. Dai, Vi-bert: Pre-training of generic visual-linguistic representations, in: 8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020, OpenReview.net, 2020.
- [20] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, M. Zhou, Layoutlm: Pre-training of text and layout for document image understanding, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2020.
- [21] Y. Xu, Y. Xu, T. Lv, L. Cui, F. Wei, G. Wang, Y. Lu, D. Florencio, C. Zhang, W. Che, et al., LayoutLMv2: Multi-modal pre-training for visually-rich document understanding, 2020, arXiv preprint arXiv:2012.14740.
- [22] C. Chen, T. Su, G. Meng, Z. Xing, Y. Liu, From UI design image to GUI skeleton: a neural machine translator to bootstrap mobile GUI implementation, in: Proceedings of the 40th International Conference on Software Engineering, ICSE 2018, 2018.
- [23] K.P. Moran, C. Bernal-Cárdenas, M. Curcio, R. Bonett, D. Poshyvanyk, Machine learning-based prototyping of graphical user interfaces for mobile apps, IEEE Trans. Softw. Eng. (2018).
- [24] E. Schoop, X. Zhou, G. Li, Z. Chen, B. Hartmann, Y. Li, Predicting and explaining mobile ui tappability with vision modeling and saliency analysis, in: Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems, 2022, pp. 1–21.
- [25] A. Swearngin, Y. Li, Modeling mobile interface tappability using crowdsourcing and deep learning, in: Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, 2019, pp. 1–11.
- [26] J. Wu, R. Krosnick, E. Schoop, A. Swearngin, J.P. Bigham, J. Nichols, Never-ending learning of user interfaces, in: Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, 2023, pp. 1–13.
- [27] S. Bunian, K. Li, C. Jemali, C. Harteveld, Y. Fu, M.S. El-Nasr, VINS: Visual search for mobile user interface design, in: The 2021 ACM Conference on Human Factors in Computing Systems, CHI2021, 2021.

- [28] J. Chen, C. Chen, Z. Xing, X. Xia, L. Zhu, J. Grundy, J. Wang, Wireframe-based UI design search through image autoencoder, *ACM Trans. Softw. Eng. Methodol. (TOSEM)* 29 (3) (2020).
- [29] T. Zhao, C. Chen, Y. Liu, X. Zhu, GUIGAN: Learning to generate GUI designs using generative adversarial networks, 2021, arXiv preprint arXiv:2101.09978.
- [30] Z. Liu, C. Chen, J. Wang, Y. Huang, J. Hu, Q. Wang, Owl eyes: Spotting UI display issues via visual understanding, in: 2020 35th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2020.
- [31] N. Cooper, C. Bernal-Cárdenas, O. Chaparro, K. Moran, D. Poshyvanyk, It takes two to tango: Combining visual and textual information for detecting duplicate video-based bug reports, 2021, arXiv preprint arXiv:2101.09194.
- [32] G. Li, G. Baechler, M. Tragut, Y. Li, Learning to denoise raw mobile UI layouts for improving datasets at scale, in: Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems, 2022, pp. 1–13.
- [33] N. Reimers, I. Gurevych, Sentence-BERT: Sentence embeddings using siamese BERT-networks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP, 2019.
- [34] R. Girshick, J. Donahue, T. Darrell, J. Malik, Rich feature hierarchies for accurate object detection and semantic segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2014, 2014.
- [35] R. Girshick, Fast r-cnn, in: Proceedings of the IEEE International Conference on Computer Vision, ICCV2015, 2015.
- [36] S. Ren, K. He, R.B. Girshick, J. Sun, Faster R-CNN: towards real-time object detection with region proposal networks, in: Advances in Neural Information Processing Systems, 2015.
- [37] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C.L. Zitnick, Microsoft coco: Common objects in context, in: European Conference on Computer Vision, Springer, 2014.
- [38] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016.
- [39] S. Xie, R. Girshick, P. Dollár, Z. Tu, K. He, Aggregated residual transformations for deep neural networks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, 2017.
- [40] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, Focal loss for dense object detection, in: Proceedings of the IEEE International Conference on Computer Vision, ICCV2017, 2017.
- [41] S. Zhang, C. Chi, Y. Yao, Z. Lei, S.Z. Li, Bridging the gap between anchor-based and anchor-free detection via adaptive training sample selection, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, 2020.
- [42] X. Zhang, F. Wan, C. Liu, X. Ji, Q. Ye, Learning to match anchors for visual object detection, *IEEE Trans. Pattern Anal. Mach. Intell.* (2021).
- [43] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, ImageNet: A Large-Scale Hierarchical Image Database, in: Proceedings of the IEEE International Conference on Computer Vision, CVPR2009, 2009.
- [44] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen, Mobilenetv2: Inverted residuals and linear bottlenecks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, 2018.
- [45] J. Wu, X. Zhang, J. Nichols, J.P. Bigham, Screen parsing: Towards reverse engineering of ui models from screenshots, in: The 34th Annual ACM Symposium on User Interface Software and Technology, 2021, pp. 470–483.
- [46] C. Bai, X. Zang, Y. Xu, S. Sunkara, A. Rastogi, J. Chen, et al., Uibert: Learning generic multimodal representations for ui understanding, 2021, arXiv preprint arXiv:2107.13731.
- [47] Y. Li, G. Li, X. Zhou, M. Dehghani, A. Gritsenko, VUT: Versatile UI transformer for multi-modal multi-task user interface modeling, 2021, arXiv:2112.05692.



Jingwen Fu received the B.S. degree from the School of Electronic and Information Engineering, Xi'an Jiaotong University, Xi'an, China, in 2021, where he is currently pursuing the Ph.D. degree. His research interests include learning theory, machine learning, computer vision.



Xiaoyi Zhang received the B.S. degree from the School of Software Engineering, University of Electronic Science and Technology of China, Chengdu, China, in 2021. He is now Research Software Engineer at Microsoft Research Asia. His research interests include computer vision and UI understanding.



Yuwang Wang received the B.S. degree from Department of Automation, Tsinghua University, China, in 2011, and PhD degree in control science and engineer from Tsinghua University. He is a Assistant Researcher with Beijing National Research Center for Information Science and Technology, Tsinghua University. His research interest includes representation learning and generative models.



Wenjun (Kevin) Zeng received his B.E. degree in 1990, M.S. degree in 1993, and Ph.D. degree in 1997, all in Electrical Engineering, from Tsinghua Univ., the Univ. of Notre Dame, and Princeton Univ., respectively. He is on the Editorial Board of International Journal of Computer Vision, was an Associate Editor-in-Chief or Associate Editor for more than 10 IEEE journals, and has served as the General Chair or TPC Chair for a dozen IEEE Conferences (e.g., ICME'2018, ICIP'2017). He is a Fellow of the IEEE.



Nanning Zheng (Fellow, IEEE) received the graduation degree from the Department of Electrical Engineering, Xi'an Jiaotong University, Xi'an, China, in 1975, and the M.S. degree in information and control engineering from Xi'an Jiaotong University in 1981, and the Ph.D. degree in electrical engineering from Keio University, Yokohama, Japan, in 1985. He joined Xi'an Jiaotong University in 1975, where he is currently a Professor and the Director of the Institute of Artificial Intelligence and Robotics. His research interests include computer vision, pattern recognition, and machine learning. Prof. Zheng is the Chinese Representative on the Governing Board of the International Association for Pattern Recognition. He also serves as the President of the Chinese Association of Automation. He became a member of the Chinese Academy of Engineering in 1999.